

HPC Lab Final Report: 鲲鹏 920F TRSM

- 1 实验目标 2
- 2 问题定义与评测方法 2
 - 2.1 TRSM 计算结构 2
 - 2.2 正确性与性能指标 2
- 3 实验环境与测试规模 2
- 4 Baseline 与优化思路 3
- 5 最终实现 3
 - 5.1 按形状选择递归或块算法 3
 - 5.2 对角块求解与向量化 4
 - 5.3 尾部更新与线程方向 4
 - 5.4 OpenMP 与 KBLAS 的组合 4
- 6 正确性与性能结果 5
- 7 尝试、取舍与局限 5
 - 7.1 SME 候选路径的取舍 5
 - 7.2 结构变体的系统否决 6
 - 7.3 当前结果的局限 6
- 8 可复现交付 6
- 9 总结 7

1 实验目标

本实验面向鲲鹏高性能计算全球挑战赛 S2 赛季的 TRSM 赛题，在国家超级计算深圳中心的鲲鹏 920F 节点上实现并优化双精度下三角矩阵求解。给定下三角矩阵 L 和右端矩阵 B ，程序求解

$$LX = B, \quad L \in \mathbb{R}^{m \times m}, \quad X, B \in \mathbb{R}^{m \times n} \quad (1)$$

目标是在不改变题目接口、计时方式和正确性判定的前提下，针对三种形状不同的测试规模选择合适的分块、递归和 OpenMP 并行策略。最终交付版本需要能够在目标节点上由一个脚本完成编译和测试，并对每个测试用例给出误差与吞吐量。

2 问题定义与评测方法

2.1 TRSM 计算结构

本题的 L 为下三角矩阵，求解过程存在沿行方向的前向依赖。将 L 按行块划分后，当前对角块可以先完成三角求解，再使用已经求出的上方右端块更新剩余部分：

```
for each diagonal block i:
    solve L[i:i+bs, i:i+bs] * X[i:i+bs, :] = B[i:i+bs, :]
    B[i+bs:, :] -= L[i+bs:, i:i+bs] * X[i:i+bs, :]
```

其中对角块求解是串行依赖最强的部分，尾部更新则是矩阵乘法，适合调用 KBLAS 的 DGEMM。该分解把一个整体的三角求解转化为“对角小块求解加尾部矩阵更新”的重复过程，也为根据矩阵形状选择并行方向提供了接口。

2.2 正确性与性能指标

官方 bench 首先生成随机下三角矩阵 L 和随机参考解 X_{true} ，构造 $B = LX_{\text{true}}$ ，再调用待测函数恢复 X 。误差采用结果与参考解之差的无穷范数：

$$e_{\max} = \max_{i,j} |X_{i,j} - X_{\text{true},i,j}| \quad (2)$$

本题正确性阈值为 10^{-12} 。性能代码使用 m^2n 作为计量 FLOPs，并将多次调用的平均时间换算为 GFLOPS。测试程序本身没有修改误差阈值、随机数生成、矩阵布局或计时逻辑，因此性能结果仍与赛题 bench 的判定方式一致。

3 实验环境与测试规模

实验记录对应深超算鲲鹏集群的计算节点，架构为 AArch64，处理器为鲲鹏 920F。程序使用 GCC、OpenMP、numactl 和 KBLAS，运行时固定为 38 个 OpenMP 线程，并使用 numactl -N 1 将计算限制在一个 NUMA 节点。环境脚本将 KBLAS 头文件、动态库和 BiSheng HPCKit 的 OpenMP 运行库加入搜索路径。

用例	矩阵规模	形状特征	最终路径
Case 1	$m = 512, n = 19968$	小 m 、宽 n	递归, leaf = 48, N-split
Case 2	$m = 2432, n = 17024$	中等 m 、宽 n	递归, leaf = 32, N-split
Case 3	$m = 17024, n = 512$	大 m 、窄 n	NB = 224, M-split

Table 1: TRSM 三组正式测试规模与路径选择

测试命令沿用赛题参数顺序, 即 M 、 N 、 test_runs 。当前交付脚本默认每个用例运行一次; 如果需要降低单次计时噪声, 可以通过 TRSM_TEST_RUNS 增加重复次数, 但不会改变算法和正确性检查。

```
cd codes/lab-final/trsm
bash run.sh
```

run.sh 的实际构建命令为 $\text{gcc -O3 -mcpu=native -fopenmp bench_trsm.c trsm.c -o trsm_test -lm -l:libkblas.so.25.2.1}$, 并设置 $\text{OMP_PROC_BIND=close}$ 、 OMP_PLACES=cores 。脚本依次调用:

```
numactl -N 1 ./trsm_test 512 19968 1
numactl -N 1 ./trsm_test 2432 17024 1
numactl -N 1 ./trsm_test 17024 512 1
```

4 Baseline 与优化思路

TRSM 的性能不能只由一个固定分块参数决定。三个测试用例的 m 和 n 差异较大: Case 1 和 Case 2 的右端矩阵很宽, 尾部更新按 n 分片可以让线程获得足够的连续列工作; Case 3 的右端矩阵较窄, 如果继续沿 n 方向切分, 线程并行度会受限, 因此改为沿剩余行按 m 分片。最终实现将这一判断固化为按形状分支的 V9 路径, 并在此基础上把宽 n 更新升级为二维切分的 V14 路径。

优化过程中保留了三个原则: 第一, 矩阵运算的主体交给成熟的 KBLAS DGEMM; 第二, KBLAS 在每次调用中固定为单线程, 避免其内部线程与外层 OpenMP 嵌套造成过量线程和资源争用; 第三, 只在足够大的更新上建立 OpenMP 并行区, 小矩阵更新直接串行调用 KBLAS, 减少线程创建与同步开销。

5 最终实现

5.1 按形状选择递归或块算法

正式 trsm.c 的入口是 l_trsm 。当 m 不超过 1024 时, 程序使用 leaf 为 48 的递归算法; 当 m 大于 1024 且不超过 4096 时, 使用 leaf 为 32 的递归算法; 更大的 m 使用固定块大小 224 的 right-looking 块算法。递归分割点按 32 行对齐, 使子问题和 OpenMP 分片保持规则的行边界。

```

if m <= 1024:
    recursive TRSM, leaf = 48
else if m <= 4096:
    recursive TRSM, leaf = 32
else:
    blocked TRSM, NB = 224

```

递归路径先求解左上子问题，再调用 `dgemm_update` 更新右下区域，最后求解右下子问题。大规模块路径沿对角块从上到下推进，在每个块之后更新剩余行。两条路径都保持同一个数学递推，因此只改变了任务组织方式，没有改变结果语义。

5.2 对角块求解与向量化

对角块求解按右端矩阵的列方向划分任务。对于 n 不大的用例，每个任务处理 16 列；对于宽矩阵，每个任务处理 32 列。每一行的前向代入先把当前列块读入局部累加数组，再沿已经求解的行进行乘减，最后乘以对角元素的倒数。内层列循环使用 OpenMP `simd` 指令，使编译器能够将连续列上的乘减和缩放映射到 ARM 向量指令。

这种安排保留了行之间不可消除的三角依赖，同时将不同右端列之间的独立性暴露给 OpenMP。相邻两行同时计算的 ILP 候选在节点实测中对 Case 3 产生 7.0% 的负收益，因此正式版本保留单行依赖路径。

5.3 尾部更新与线程方向

尾部更新的数学形式为

$$B_2 \leftarrow B_2 - L_{21}B_1 \quad (3)$$

实现根据更新规模在两种并行方向之间选择：当 n 大于 4096 时，沿列方向分片，每个线程处理连续的 n 列；否则沿行方向分片，每个线程处理连续的剩余行。每个线程调用单线程 KBLAS DGEMM，并写入互不重叠的 B2 区域。这样既能复用 KBLAS 的高效矩阵乘内核，又能避免多个外层线程同时写同一输出元素。

在纯列分片下，38 个线程都要完整读取一遍乘数面板 L_{21} ，该面板的重复读取达到 38 份。V14 将宽 n 更新改为二维切分：38 个线程划分为 2 个行组乘 19 个列组，行组内的 19 个线程共享同一份行区间的 L_{21} ，面板重复读取降到 19 份；代价是 B 面板的重复读取翻倍，但每个线程获得的列条带宽度也翻倍，流式访问效率更高。节点探针显示，Case 2 顶层更新形状在此切分下从约 1684 GFLOPS 提升到约 1835 GFLOPS。该路径由 `TRSM_WIDE_2D` 宏控制，仅在 n 大于 4096 且更新行数不低于 256 时启用，更深的递归小更新仍走纯列分片，Case 3 的行分片不受影响。

当 `rows`、`n`、`bs` 三者乘积较小时，代码不建立 OpenMP 并行区而直接调用 KBLAS。这一阈值策略用于抑制递归尾部和最后一个小尾块的 `fork/join` 成本，尤其适合 Case 1 的小对角块和 Case 3 的末尾块。

5.4 OpenMP 与 KBLAS 的组合

KBLAS 通过构造函数调用 `BlasSetNumThreads(1)`，把内部 DGEMM 固定为单线程。大更新由外层 OpenMP 负责分片，形成以下层次：

层次	组件	作用
线程配置	OMP_NUM_THREADS=38	使用一个 NUMA 节点上的固定线程数
任务分片	外层 OpenMP	按列或按行切分独立的 DGEMM 子任务
数值内核	单线程 KBLAS	执行每个子矩阵乘，避免嵌套并行
数据放置	numactl -N 1	限制内存和计算的 NUMA 范围

Table 2: 最终版本的并行层次

6 正确性与性能结果

以下数据取自 codes/lab-final/trsm/README.md 中的最近正式结果记录。测试程序对三组输入均先完成参考解复算，再进行性能计时；所有用例的最大绝对误差都低于 10^{-12} ，因此通过正确性门槛。

用例	规模 $m \times n$	平均时间 / ms	GFLOPS	最大绝对误差	校验
Case 1	512×19968	8.23	636.0313	$6.66e-16$	PASS
Case 2	2432×17024	77.24	1303.6841	$8.88e-16$	PASS
Case 3	17024×512	134.89	1100.0665	$1.83e-15$	PASS

Table 3: TRSM V14 正式结果记录（节点 cn22993, test_runs=5）

三组用例的显示时间总和为 220.36 ms，按 bench 使用的 FLOPs 计量，三个用例合计约 254.311137280 GFLOP，综合吞吐量约为 1154.1 GFLOPS。同批同节点的 V9 交错对照为 8.47 ms、81.83 ms、136.13 ms，综合 1123.4 GFLOPS，V14 相对提升约 2.7%，其中 Case 2 提升 5.9%，Case 1 提升 2.9%，Case 3 持平。Case 2 的吞吐量最高，原因是其 m 和 n 都足够大，递归产生的更新任务和 KBLAS DGEMM 都能得到较好的摊销；二维切分减少 L_{21} 面板重复读取的收益也在该用例上最显著。Case 1 的 m 较小，三角求解与并行区管理成本占比更高；Case 3 虽然总计算量大，但 $n = 512$ 较窄，性能更容易受到行分片、同步和内存访问的影响。

指标	结果	结论
正确性	3/3 PASS, 最大误差 1.83×10^{-15}	满足 10^{-12} 阈值
平均时间总和	220.36 ms	三组正式用例合计耗时
综合吞吐量	1154.1 GFLOPS	按 bench 的 m^2n 计量
同批对照	V9 综合 1123.4 GFLOPS	V14 相对提升约 2.7%
目标对比	未达到 3000 GFLOPS	引擎级上限已被量化，见下节

Table 4: 正式结果摘要

7 尝试、取舍与局限

7.1 SME 候选路径的取舍

实验记录中对普通 SVE、SME FMOPA 以及自有更新内核做过能力和微内核验证。节点具有 SVE2、SME 和 smef64f64 特征，实测 SVE 向量长度为 512 bit；但 HWCAP2_SVEF64MM 为 0，不能把 FMMLA 作为可靠的通用路径。KBLAS 的 DGEMM 反汇编包含 SME 的 FMOPA 指令，说明现有库已经能够使用 SME F64F64 路径。

自有 16×32 SME 更新内核在真实 load/store 条件下约为 1737 GFLOPS，加入共享 B 打包和每线程 A 打包后约为 1525 GFLOPS。分段计时修正后的 KBLAS 更新段真实速率约为 1500 GFLOPS，自有 SME 内核相对该速率没有实质优势，还需要额外的打包、尾块和线程协作逻辑，最终没有把 SME 候选接入正式 run.sh。这是一个基于端到端风险和实测结果的取舍，提交版本优先保留可复现、可验证的 KBLAS 路径。

7.2 结构变体的系统否决

在 V14 之前，还系统测过六个结构性变体，全部通过正确性检查但在同批交错对照中负收益：k 聚合（把 Case 3 的四步更新合并为 $k=896$ 的一次大更新，破坏 B 面板的 L2 驻留，Case 3 慢 2.5%）；扁平分块替代递归（宽 N 用例的更新 k 变小，慢 35% 到 40%）；持久区域加固定列条带（同样受制于小 k 更新，慢 60%）；强制 M-split 用于宽 N（每线程行数太瘦，慢 66%）；对角求解双行 ILP（Case 3 慢 7.0%）；Case 3 的求解与更新流水线重叠（求解组占用线程造成的更新组饥饿大于重叠收益，最好情形仍慢 2%）。这些负结果与 V14 的正收益一起，说明当前算法结构已处于该引擎下的较优位置。

7.3 当前结果的局限

当前记录的综合吞吐量距离 3000 GFLOPS 目标仍有差距，且该差距的大部分已被量化为结构性上限。Case 2 即使所有更新都按最优速率均匀执行，更新段下限约 54.6 ms 加求解 12.1 ms 为 66.7 ms；Case 3 的更新段单独就需要约 111 ms，加上求解与同步至少 133 ms。两 case 下限之和 199.7 ms 已超过 1800 GFLOPS 所需的 141.3 ms 总预算，因此在 KBLAS 引擎下 1800 GFLOPS 综合不可达，V14 的 1154.1 GFLOPS 已接近该引擎可达上限。主要限制包括：

1. 对角块求解仍具有严格的行依赖，OpenMP 只能利用右端列方向的并行性；
2. Case 3 的右端矩阵较窄，M-split 虽然提高了并行度，但也增加了线程间同步和不连续矩阵分片的压力；
3. 每个大更新都需要建立外层 OpenMP 区域，递归路径中的小任务无法完全摊销线程管理成本；
4. 目前正式路径依赖 KBLAS 的单线程 DGEMM，进一步提升需要在不破坏精度的情况下验证更细粒度的库线程协作或专用内核。

这些局限说明，继续优化时应以三组正式规模的端到端交错测试为准，而不能只依据某一个独立 tile 微基准的峰值。任何新路径都需要同时通过误差、尾块和线程缩放检查，再与 V14 进行同节点对照。

8 可复现交付

报告对应的提交目录为 codes/lab-final/trsm/，当前只保留提交所需文件：trsm.c、bench_trsm.c、README.md、env.sh、run.sh 和 trsm_submission.zip。其中：

1. trsm.c 提供最终 V14 l_trsm 实现；
2. bench_trsm.c 提供随机矩阵生成、参考解构造、误差检查和计时；
3. env.sh 配置目标节点上的 KBLAS 和 OpenMP 运行时；
4. run.sh 完成编译，并按三个正式规模依次运行测试；
5. trsm_submission.zip 是提交代码和脚本的压缩包。

在目标节点上执行下面的命令即可重建测试程序并运行三组测试：

```
cd codes/lab-final/trsm
TRSM_TEST_RUNS=1 bash run.sh
```

本报告的数值表只采用已有正式记录，没有把未达标 SME 候选的微基准结果混入最终吞吐量，也没有为缺少原始终端截图的部分虚构截图或额外性能数字。

9 总结

本实验针对下三角求解的形状差异，构建了递归和块算法结合的混合 TRSM 实现。小、中规模使用不同 leaf 的递归路径，大规模使用 224 行块并沿 M 方向分片；宽右端矩阵的更新在 V14 中升级为 2 行组乘 19 列组的二维切分，把乘数面板的重复读取从 38 份降到 19 份。对角求解通过连续列 SIMD 化，尾部更新交给单线程 KBLAS DGEMM，并由外层 OpenMP 提供任务级并行，从而避免嵌套 OpenMP 的线程争用。

最终三组测试全部通过 10^{-12} 正确性阈值，最大误差为 1.83×10^{-15} ，综合吞吐量为 1154.1 GFLOPS，相对 V9 基线提升约 2.7%。分段下限的求和分析进一步表明，该结果已接近 KBLAS 单线程 DGEMM 引擎在此节点上的可达上限；继续逼近更高目标需要引入非 KBLAS 的更新内核，而这已被端到端实测证明收益有限。